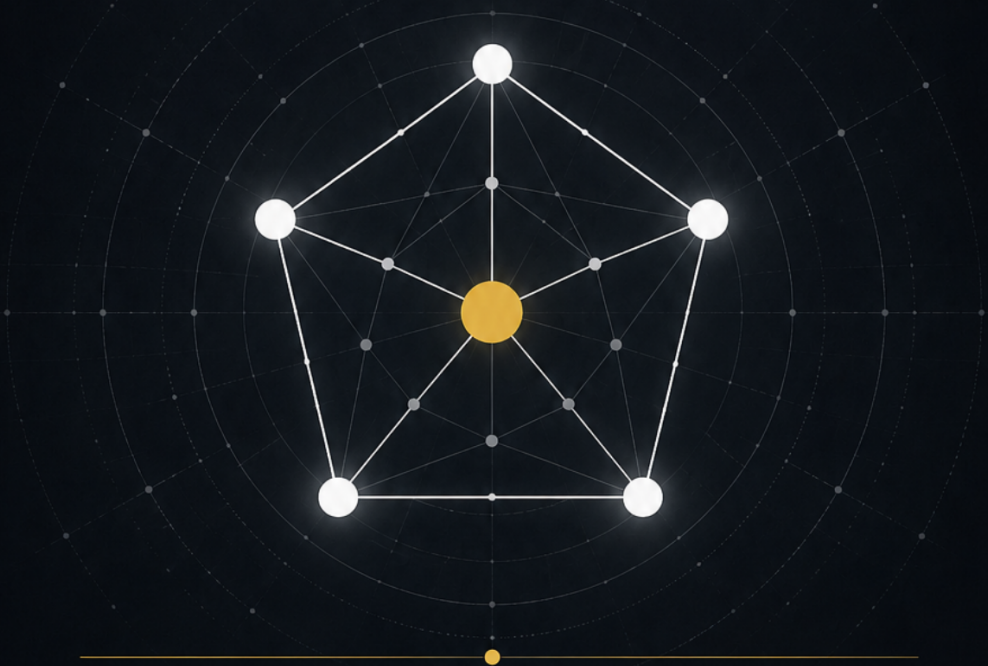
CÓDIGO SINTÉTICO

El arte de orquestar agentes de IA: arquitectura, economía y criterio de ingeniería en la era multiagente





Una guía integral para diseñar, coordinar y escalar sistemas multiagente con inteligencia, eficiencia y propósito.

SERGIO PEREZ RUIZ

Muestra gratuita

Este archivo es un fragmento de «Código sintético: el arte de orquestar agentes de IA», de Sergio Perez Ruiz.

Incluye el índice completo, el prefacio, el manifiesto y el capítulo 1 íntegro («El despertar agéntico»), más el inicio de cuatro capítulos centrales.

La obra completa tiene 191 páginas y 21 capítulos sobre arquitectura, economía y criterio de ingeniería para sistemas multiagente en producción.

Los cuatro volúmenes de la serie (ES/EN) siguen siendo de lectura abierta bajo licencia CC BY-NC-SA 4.0.

Obtener el libro completo (Kindle, tapa blanda, tapa dura)

© 2026 Sergio Perez Ruiz · ISBN 978-612-03-2931-3 (PDF) · Depósito Legal BNP N.º 2026-0963

Índice

Página legal 1

Dedicatoria 3

Sobre el autor 4

El camino a este libro 5

Obra pública destacada del autor 6
Perfiles y recursos adicionales 6

Agradecimientos 7

Créditos 8
Herramientas de composición y compilación 8
Entornos de edición 8
Verificación técnica de código y ejemplos 9
Diseño y diagramas de arquitectura 9
Herramientas de validación editorial 9
Recursos y referencias técnicas 9
Organización del libro 10
Convenciones tipográficas 10
Sobre los ejemplos de código 11

Prefacio: Delegar no es diseñar 12
¿Por qué este libro? 13
A quién está dirigido 13
Lo que este libro es 13

Manifiesto 15
Antes 15
Ahora 15
El puente de la Ingeniería de Software 15

1 El despertar agéntico 17
El desarrollador actual 17

Qué es, formalmente, un agente 18
Reactivo y proactivo: las dos mitades de la agencia 18
La proactividad ambiental 19
Marco de evaluación de riesgo y router 20
Marco de niveles de autonomía de sistemas IA 21
Ejercicio para clasificar tus sistemas 25
Lo esencial del capítulo 1 26

2 El AI Engineer 28
Un rol emerge 28
La matriz de diferenciación 28
Las siete competencias 29
El camino de conversión: de developer a AI Engineer 34
Ejercicio resuelto: diseñe el programa de conversión 35
Lo esencial del capítulo 2 35

3 Platform Engineering en la era agéntica 37
La plataforma como producto interno 37
El AI Gateway: componente crítico 37
Plantillas de equipo 40
Matriz de arquitectura según el tamaño del proyecto 41
El costo cognitivo de la deuda que no ves en la factura 44
Ejercicio resuelto que añade corte por proyecto 47
Arquitectura asíncrona como sistema de eventos 47
Límite de tokens en enjambres de agentes 51
Lo esencial del capítulo 3 56
Puente al Capítulo 10: cuándo migrar a Kafka 56

4 Patrones fundamentales 59
De la cadena de texto al sistema cognitivo 59
El ciclo ReAct que atiende el razonamiento que actúa 61
Tree of Thoughts (ToT) con bifurcación y poda de hipótesis . . 63
El coste de la exploración heurística 63
Reflexion es la autocrítica evaluada 66
La necesidad de un evaluador objetivo 67
Comparativa de patrones fundamentales 67
La evolución hacia sistemas de ingeniería 68
Lo esencial del capítulo 4 72

5 El arnés del agente 74
El código que el proveedor no te da 74
Componentes del arnés 74
Trazabilidad y registro estructurado 76

Lo esencial del capítulo 5 80

6 La variable de confianza 81
La fórmula que faltaba 81
La ecuación reformulada 81
La variable de verificabilidad 83
Patrón Shadow Agent 85
Lo esencial del capítulo 6 86

7 RACI en enjambres 87
La paradoja de la claridad 87
La regla cardinal 87
Eisenhower aplicado para delegar a un agente 88
Ejercicio resuelto de ajuste dinámico 91
Lo esencial del capítulo 7 92

8 SOLID para agentes 93
Por qué SOLID es gestión de complejidad 93
Los 5 principios reescritos 94
Refactoring de enjambre en 4 pasos con checklist real 95
Lo esencial del capítulo 8 97

9 Invariantes de producción 98
Los 5 invariantes que todo sistema debe satisfacer 98
La verificabilidad es igual a la nueva confianza 98
El costo real es igual a la deuda de verificación 99
La autonomía se gana y no se asigna 99
El contexto es capital y no es gratis 100
La arquitectura domina al modelo 101
Caso de estudio de un despliegue que falló 102
Lo esencial del capítulo 9 103

10 DDD + Kafka como sistema nervioso 104
Bounded contexts para agentes 104
Lo esencial del capítulo 10 106

11 Gobernanza de deuda técnica agéntica 107
Por qué la deuda agéntica se acumula más rápido 107
Métricas automáticas para detectar deuda 107
Revisiones de arquitectura que incluyen output de agentes . . . 108
CI/CD para bloquear deuda agéntica 109
Caso de estudio de un deploy sin validación 112
Lo esencial del capítulo 11 113

12 Revisión de código con agentes **114**
Matriz RACI para PR review 114
El agente observa bugs que pasamos por alto 114
Caso de estudio de reducción de bugs con agentes 115
Patrones para incorporar personas en el loop 116
Ejercicio resuelto que integra review agent en CI/CD 116
Lo esencial del capítulo 12 117

13 Triaje de producción y debugging autónomo **119**
Cuándo permitir que un agente depure en vivo 119
El playbook 120
Caso de estudio de un incidente resuelto por un agente 120
El agente falla sin un contexto adecuado 122
Lo esencial del capítulo 13 123

14 Generación de documentación y conocimiento **124**
Por qué la documentación generada es mejor que la manual . . 124
El riesgo de documentación que parece real y no lo es 124
Documentación de decisiones arquitectónicas **ADRs** 125
Lo esencial del capítulo 14 126

15 Atención al cliente y soporte técnico **127**
Triaje conversacional con RAG 127
Caso de estudio de tickets resueltos y reducción de costos . . . 129
Lo esencial del capítulo 15 130

16 Decisiones arquitectónicas que no puedes automatizar **131**
Qué decisiones requieren criterio humano 131
Patrón: Human-in-the-loop para decisiones irreversibles 132
Ejercicio resuelto: crea un gate para la base de datos 134
Lo esencial del capítulo 16 134

17 Seguridad en sistemas multiagente **136**
La superficie de ataque se multiplica 136
Modelo de amenazas para enjambres 136
MCP como control de acceso y no solo protocolo 139
Caso de estudio del agente mal configurado 140
Lo esencial del capítulo 17 142

18 Optimización de costos real **143**
Lo que no verás en los cálculos de precio 143
Cálculo de TCO 144
Estrategias que funcionan 145
Caso de estudio de un equipo que redujo la factura 146

Lo esencial del capítulo 18 147

19 La geopolítica como restricción arquitectónica 148
De la neutralidad técnica a la soberanía digital 148
Desglose de la arquitectura de tres capas bajo restricciones geopolíticas 149
Matriz de trazabilidad: Componentes arquitectónicos vs. Dimensiones geopolíticas 151
El riesgo de la cadena de suministro 152
Tabla comparativa: USA, China, EU, LATAM 153
Diseñar para la sustituibilidad **Backend Agnóstico** 153
Caso de estudio del equipo atado a un proveedor 155
Lo esencial del capítulo 19 157

20 El proyecto y sus patrones 159
Cuándo un agente es innecesario y un script basta 159
Cuándo un workflow supera a un sistema multiagente 160
Cuándo la arquitectura cuesta más de lo que ahorra 160
El costo de supervisión humana y gobierno organizacional . . . 161
Los límites del backend agnóstico 162
La evaluación cuantitativa de modelos 165
Marco de decisión basado en evidencia 167
Escalar la caché semántica sin destruir la latencia 168
Cómo evitar la sobreingeniería en proyectos de IA 168
Matriz de decisión 169
Lo esencial del capítulo 20 170

21 De código a criterio 171
Lo que permanece 171
El AI Engineer como garante y no como ejecutor 171
Los 10 principios finales del libro 172
Epílogo hacia la próxima generación 173

Repositorio de referencia 174

Tesis académicas de referencia 177

Referencias 179

Obras y publicaciones del autor 184
Artículos técnicos y ensayos 184
Publicaciones técnicas en LinkedIn 184
Proyectos y comunidades 185

Página legal

Código sintético *El arte de orquestar agentes de IA: arquitectura, economía y criterio de ingeniería en la era multiagente*

© 2026 Sergio Perez Ruiz. Todos los derechos reservados.

Ficha Técnica Editorial:

- **Autor-Editor (Persona Natural):** Sergio Perez Ruiz
- **Documento de identidad:** DNI N° 46581382 — Titular legal de la obra y de los derechos de publicación
- **Dirección del autor-editor:** Jr. Juan Guillermo Moore 357, Santa Anita, Lima, Perú
- **Contacto y licencias:** sergio@specsolid.com
- **Lugar de edición:** Lima, Perú
- **Fecha de publicación:** Primera edición, agosto de 2026

Registro de Propiedad Intelectual e Identificación:

- **ISBN (Edición digital / EPUB):** 978-612-03-2914-6
- **ISBN (Edición digital / PDF):** 978-612-03-2931-3
- **Hecho el Depósito Legal en la Biblioteca Nacional del Perú Nº:** 2026-09631

Formato y distribución:

Esta obra se publica exclusivamente en formato digital (EPUB y PDF) mediante impresión bajo demanda a través de las siguientes plataformas autorizadas:

- **Amazon Kindle Direct Publishing (KDP)** — distribución global en formato Kindle / EPUB
- **Google Play Books** — distribución en la plataforma de Google
- **Lulu** — edición digital y opción de impresión bajo demanda a pedido del lector

- Otras plataformas de distribución digital autorizadas por el autor-editor

No existe tiraje físico definido de fábrica. La impresión física, de estar disponible, se realiza exclusivamente bajo pedido (*print-on-demand*) a través de las plataformas mencionadas, sin stock físico mantenido por el autor-editor.

Ninguna parte de esta publicación puede ser reproducida, almacenada en un sistema de recuperación de información ni transmitida en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopiado, grabación u otro) sin el permiso previo y por escrito del autor-editor, salvo en los casos de citas breves incorporadas en reseñas críticas y ciertos usos no comerciales permitidos por la Ley sobre Derecho de Autor (Decreto Legislativo Nº 822 y Ley Nº 31250 del Perú).

Las marcas comerciales mencionadas en este libro (Claude, Anthropic, GPT, OpenAI, Gemini, Google, Python, GitHub, entre otras) son propiedad de sus respectivos titulares. Su mención es únicamente con fines educativos e informativos y no implica afiliación ni patrocinio.

Aviso de responsabilidad: La información contenida en este libro se ofrece «tal cual». Aunque el autor ha tomado todas las precauciones razonables en su preparación (incluida la verificación de cada referencia bibliográfica y la validación técnica de los ejemplos de código), ni el autor ni el editor asumen responsabilidad alguna por errores u omisiones, ni por daños resultantes del uso de la información aquí contenida. Los precios de las APIs, identificadores de modelos, *rankings* comparativos y capacidades técnicas corresponden a un **snapshot fechado en agosto de 2026** y cambian con frecuencia: cada tabla y cifra de este tipo se presenta junto con su fórmula de origen para que pueda recalcularse con precios vigentes. Verifique siempre la documentación oficial del proveedor, o el catálogo vivo de modelos y precios del repositorio complementario (`github.com/sergioide007/synthetic-code`, módulo `src/sintetico/model_registry.py`, sobreescribible por variables de entorno), antes de tomar decisiones de arquitectura o presupuesto.

Sobre el autor

Sergio Perez Ruiz es ingeniero de sistemas y especialista en sistemas de inteligencia artificial aplicada. Su trabajo se centra en el diseño de plataformas agénticas de producción, la economía de los sistemas basados en modelos de lenguaje y la formación de equipos de ingeniería para la era de la orquestación. Este libro condensa esa experiencia práctica con el rigor de la literatura académica revisada por pares, conectando principios clásicos de ingeniería de software con los nuevos desafíos introducidos por los sistemas basados en inteligencia artificial.

Prefacio: Delegar no es diseñar

> *La ingeniería de software no desaparece con los agentes. Evoluciona.* — del manifiesto de esta obra.

Hacia 2026, el software atravesó una transformación tectónica. La IA dejó de ser una herramienta para «copiar y pegar código con mejor sintaxis» y se convirtió en un sistema capaz de razonar, descomponer, delegar y autoevaluarse. La literatura técnica le dio un nombre: *agencia* (Wooldridge 2009; Anthropic 2024). La evidencia llegó antes que los marcos: un experimento con GitHub Copilot encontró que los desarrolladores asistidos completaban tareas un 55,8 % más rápido (Peng et al. 2023), y estudios posteriores confirmaron mejoras sostenidas (Ziegler y Donkers 2024).

Pero esos estudios medían *asistentes*. Los sistemas *agénticos* —donde la IA ejecuta ciclos completos de percepción, razonamiento y acción sin supervisión constante— multiplican promesa y riesgo por igual (Kapoor et al. 2024). Y ahí es donde casi todos los equipos que conozco tropiezan de la misma forma: confunden velocidad de generación con capacidad de entrega.

Este libro no celebra esa velocidad. Advierte sobre sus consecuencias cuando no hay un marco de diseño detrás. Cuando delegamos el desarrollo a inteligencias artificiales sin arquitectura ni métricas claras, ocurren tres fenómenos, siempre en este orden:

- **El *Token-Burn* silencioso.** El costo crece superlinealmente mientras la calidad se estanca. Cada llamada, vista de forma aislada, parece trivial. El problema nunca se ve línea por línea: emerge a escala, en la factura del mes.
- **La deuda técnica acelerada.** Un agente que ignora el contexto del negocio genera lógica inválida a velocidad industrial. La deuda que un equipo humano acumulaba en un año, un sistema agéntico mal gobernado la produce en una semana (Sculley et al. 2015).
- **La ilusión de velocidad.** Más código en menos tiempo, pero código que no supera pruebas ni revisión. Las «líneas generadas» suben en el dashboard. Las funcionalidades que realmente llegan a

producción, no. Los tres fenómenos comparten una misma raíz: se trató la orquestación de agentes como un problema de *prompting*, cuando en realidad es un problema de ingeniería de sistemas.

¿Por qué este libro?

La solución no pasa por ralentizar, sino por *orquestar*: sistemas donde los agentes se revisan entre sí, debaten, se alinean con el negocio y rinden cuentas de cada decisión que toman de forma autónoma. La evidencia respalda el enfoque: los sistemas multiagente producen respuestas más factuales que un único modelo trabajando solo (Du et al. 2023), y superan de forma consistente a los agentes monolíticos en tareas complejas (Hong et al. 2024; Qian et al. 2024).

Ese es el arte que este libro enseña: no *si* delegar en agentes, sino *cómo* construir la arquitectura, la economía y el gobierno que hacen que delegar sea seguro, medible y rentable. Por eso el libro no termina en un capítulo de arquitectura ni en uno de costos. Termina en un capítulo sobre criterio —el juicio de ingeniería que decide, en cada nivel de autonomía, cuánto delegar y cuánto retener. Las herramientas cambiarán; ese criterio es lo único que se queda contigo de un modelo al siguiente.

A quién está dirigido

Al **ingeniero senior** que domina la ingeniería clásica y quiere aprender las primitivas agénticas sin abandonar lo que ya sabe que funciona. Al **arquitecto** que debe construir una plataforma interna para que cincuenta equipos usen IA sin quemar el presupuesto ni el uptime. Al **directivo técnico** que necesita separar el ruido del marketing de la evidencia operativa, y tomar decisiones de inversión con datos, no con entusiasmo.

Lo que este libro es

Es un manual de ingeniería, donde descubrimos que la disciplina convierte a una tecnología poderosa en una tecnología útil. El modelo importa, pero aún más importante es el criterio, pues te convierte en alguien capaz de operar esta tecnología en producción.

Un mensaje para quien diseña la plataforma

Es muy probable que hoy ya estés intentando construir un arnés o una arquitectura técnica con todo lo necesario para tu caso de uso. Incluso es posible que, al incorporar los conceptos y técnicas de estas páginas, desarrolles soluciones aún mejores y más adaptadas a tu contexto particular.

La premisa es simple: lo que hoy construyes con criterio técnico perdura y se convierte en una base sólida. Esto aplica tanto para el arquitecto individual como para los equipos de ingeniería de software que necesitan fijar sus estándares sobre un terreno firme, auditable y escalable.

Sergio Perez Ruiz, 2026

Manifiesto

Los agentes desarrollan código muy rápido, muy bueno y eficaz; incluso te dicen dónde está el error y ejecutan las pruebas.

Antes

Cuando iniciaba en el mundo de la programación y me adentraba en el desarrollo de sistemas de información, hace más de una década, algo me llamaba la atención: la complejidad con la que desarrollábamos. Invertíamos mucho tiempo en revisar; incluso, algunas veces, por querer aprender más, me quedaba desarrollando más tiempo, creyendo que alcanzaría un buen nivel pronto. Luego, con la ayuda de un buen amigo y maestro, aprendí que la clave no era ir tras la moda o lo último en código, sino dominar la arquitectura, los principios de diseño y el rigor metodológico.

Ahora

Hoy en día no necesitamos escribir cada línea de código de forma manual. Sin embargo, como los agentes producen comportamientos no especificados línea por línea, el trabajo del ingeniero de software evoluciona: nos enfocamos en diseñar las condiciones, los límites, las invariantes y las arquitecturas sobre los cuales estos sistemas autónomos pueden operar de forma segura y consistente.

El puente de la Ingeniería de Software

Unificamos el ciclo de vida (SDLC) y la arquitectura (Runtime) en una matriz de transición compacta:

Capa / Fase	Software Clásico	Puente de Integración	Sistemas Multiagente
1. Entidades	Requisitos, Clases y Servicios	Esquemas e Interfaces	Roles, Metas e Invariantes
2. Interacción	Monolitos/Micro, REST y RPC	Middlewares y Brokers	Redes, ACL y Negociación
3. Decisión	Algoritmos y Código OOP	APIs y Orquestadores	Razonamiento BDI y Prompts
4. Control/Riesgo	Unit Tests, DB y Transacciones	Sandboxes y Data Fabric	*Evals*, Vectores y *Guardrails*
5. Operación	CI/CD, Binarios y Observabilidad	Infraestructura distribuida	Trazabilidad y Autonomía
6. Evolución	Parches y Refactorización	Estándares de Calidad	Alineamiento y Aprendizaje

Esta matriz ilustra cómo la ingeniería clásica evoluciona en seis capas hacia el paradigma agéntico a través de su puente de integración. De ella se desprende nuestra máxima: la ejecución técnica del código se delega como tarea operativa, pero la arquitectura, la gobernanza y los límites se ejercen activamente como acto de autoridad sobre el sistema. Aunque los agentes prometen agilidad, la complejidad no ha desaparecido; por el contrario, aumentan la complejidad accidental y las dependencias de proveedores cuyos modelos responden a sus propios intereses. Por esta razón, delegar sin ejercer produce software opaco, frágil y supeditado a terceros, mientras que ejercer antes de delegar da lugar al código sintético que necesitamos: un software seguro, alineado con el negocio, rápido en su ejecución y robusto en su diseño.

> Código Sintético te ayuda a eso: a retomar el rumbo hacia la transformación digital agéntica, porque como me lo enseñó mi maestro, a quien citaré con mucho aprecio: *la única forma de ir rápido es ir bien.* (Martin 2017)

Nota: Aunque Robert C. Martin popularizó masivamente esta máxima en la industria del software (*«The only way to go fast, is to go well»*) a través de *Clean Architecture* y *Clean Code*, el propio autor acredita que la premisa proviene de los principios de diseño y gestión de sistemas clásicos (en particular, de la aviación militar y del entrenamiento de respuesta rápida): *«Slow is smooth, smooth is fast»* (Lento es fluido, fluido es rápido).

— **Sergio Perez Ruiz**

El despertar agéntico

El desarrollador actual

Durante cuarenta años medimos la productividad de un programador en líneas de código por hora. Tickets cerrados por sprint. Pull requests integrados por semana. Métricas industriales aplicadas a un trabajo puramente intelectual. Había un techo físico: la velocidad de escritura en el teclado. Ese techo desapareció. No ocurrió en un año específico; fue una evolución en cadena:

- **2017:** Transformer (Vaswani et al. 2017). La arquitectura que lo hizo todo posible.
- **2020:** GPT-3 (Brown et al. 2020). *Few-shot learning*. El modelo resolvía tareas nuevas sin necesidad de reentrenamiento.
- **2021:** Codex (M. Chen et al. 2021). Un modelo especializado capaz de escribir código con un éxito que sorprendió a sus propios creadores.
- **2022:** RLHF (Ouyang et al. 2022). Ajuste mediante retroalimentación humana. Emergencia de capacidades operativas visibles de inmediato.
- **2023:** ReAct (Yao, Zhao, et al. 2023). Los modelos aprendieron a actuar, a usar herramientas externas y a alternar razonamiento con ejecución.
- **2024-2026:** Sistemas multiagente basados en LLM (S. Chen et al. 2024). Arquitecturas con múltiples agentes especializados, coordinación de tareas y supervisión humana para aplicaciones complejas.

El desarrollador actual ya no escribe sintaxis desde cero; opera herramientas que la automatizan. La destreza técnica no desaparece, pero el cuello de botella físico se elimina. Pasamos de programar código manualmente a gobernar el flujo de código generado ya no por nuestras

propias capacidades intelectuales, sino mediante la ayuda de los modelos de inteligencia artificial.

Qué es, formalmente, un agente

Un agente no es un chatbot. La industria confunde este límite constantemente debido al marketing. La definición formal viene de la inteligencia artificial distribuida y es muy anterior a los LLMs. Wooldridge la planteó en 2009, antes de los Transformers, de GPT y de que existiera el término *prompt*. Su definición sigue siendo exacta: > Un agente es una entidad situada en un entorno, capaz de percibir ese entorno, razonar sobre él, actuar mediante efectores y aprender de los resultados. Traducido a la ingeniería de software actual con LLMs:

- **Percibe:** Lee el contexto de la conversación, mensajes entrantes y resultados de herramientas ejecutadas.
- **Razona:** Procesa información mediante su conocimiento paramétrico combinado con el contexto inyectado en el *prompt*.
- **Actúa:** Ejecuta llamadas a funciones, herramientas locales y APIs externas.
- **Aprende:** Adapta su comportamiento dentro de la misma sesión a través del historial de contexto, no modificando los pesos del modelo.

Un chatbot se queda a mitad de camino: percibe y razona, pero no actúa sobre el sistema subyacente. No cierra el ciclo operativo.

Reactivo y proactivo: las dos mitades de la agencia

La teoría clásica descompone la agencia en una tensión constante entre dos capacidades que rara vez conviven de forma nativa (Wooldridge 2009):

- **Reactivo:** Responde a los cambios del entorno en tiempo real. El bug imprevisto, el test que se rompe, la alerta de infraestructura que salta.
- **Proactivo:** Toma la iniciativa, descompone la meta principal en subtareas, planifica y persiste hasta cumplir el objetivo.

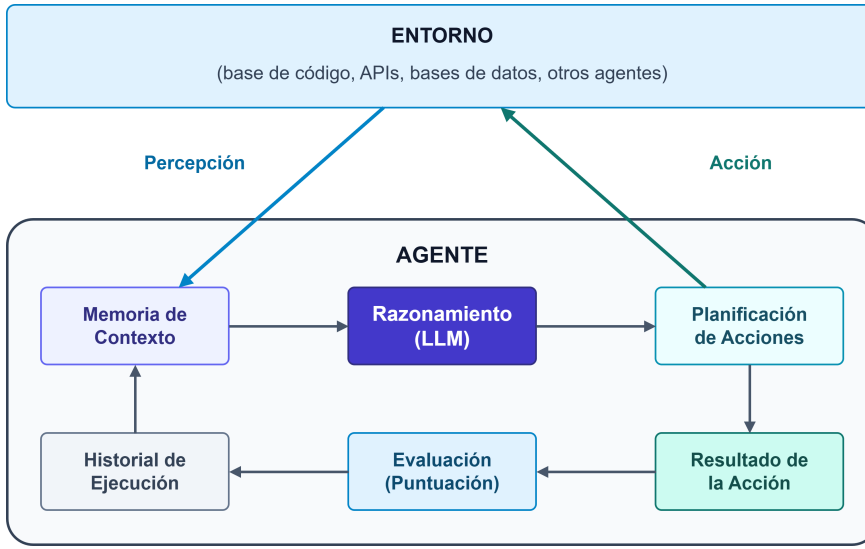


Figura 1.1: Ciclo de un agente autónomo

Un agente puramente reactivo es incapaz de emprender proyectos por sí mismo; responde rápido a estímulos, pero no resuelve tareas complejas de varios pasos. Un agente puramente proactivo diseña planes perfectos para un entorno estático, ignorando que el mundo real se mueve mientras calcula. El equilibrio técnico en producción se llama **ReAct (Reasoning + Acting)**. El patrón detallado en el Capítulo 4 intercala observación y razonamiento técnico en cada ciclo para resolver esta desconexión.

La proactividad ambiental

La tendencia actual del mercado apunta hacia los "agentes ambientales". Sistemas que corren en segundo plano, se disparan mediante eventos de arquitectura y solo nos notifican cuando detectan anomalías que requieren atención. La promesa de automatización es alta, pero exige un escrutinio estricto: cada grado de proactividad que gana el sistema es un grado de control operativo que cede la organización. Un agente ambiental que toma decisiones críticas sin supervisión ejecuta criterios que aún no has auditado en producción. *(Consúltese el Capítulo 6 para profundizar en el concepto de confianza calibrada).*

Marco de evaluación de riesgo y router

Mide la **criticidad y el costo** (*Blast Radius*) de las tareas para derivarlas a carriles con mayor o menor grado de supervisión humana (Niveles N1 a N5).

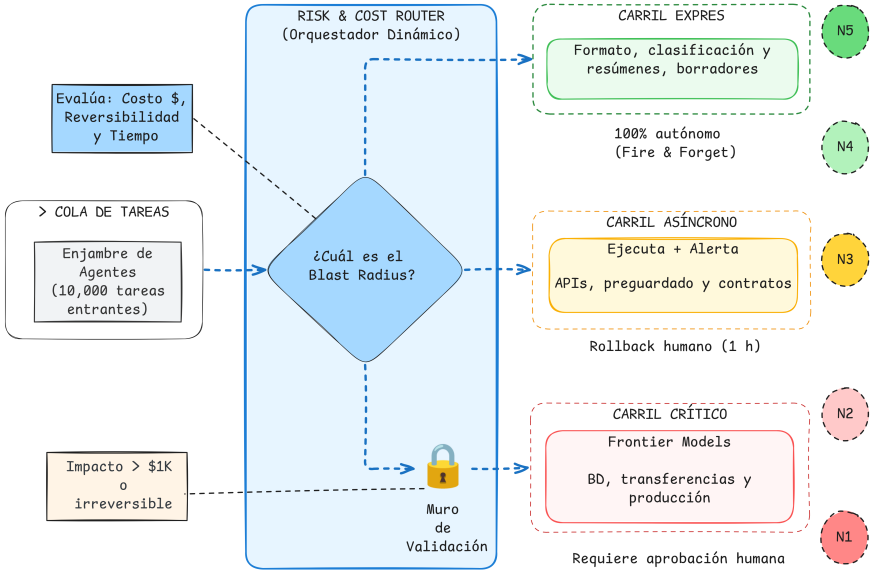


Figura 1.2: Evaluación de riesgo y router

Para representar adecuadamente el flujo del **Risk & Cost Router**, se debe emplear la siguiente matriz:

Nivel	Carril Asignado	Criterio de Riesgo / Blast Radius	Acciones / Alcance	Mecanismo de Control
N5 / N4	**Carril Exprés**	Bajo costo, alta reversibilidad y tiempo corto.	Formatos, clasificación, resúmenes y borradores.	**100% Autónomo** (*Fire & Forget*).

Nivel	Carril Asignado	Criterio de Riesgo / Blast Radius	Acciones / Alcance	Mecanismo de Control
N3	**Carril Asíncrono**	Riesgo moderado.	Ejecución + Alerta, llamadas a APIs, preguardados y contratos.	**Autónomo con opción de Rollback** (ej. ventana de 1 hora).
N2 / N1	**Carril Crítico**	Alto impacto (> $1K) o acción irreversible.	Uso de *Frontier Models*, modificación de BD, transferencias y despliegue a producción.	**Muro de Validación:** Requiere aprobación humana previa (*Human-in-the-loop*).

Marco de niveles de autonomía de sistemas IA

La agencia no es un interruptor binario; es un espectro gradual de autonomía. Definimos cinco niveles operativos para determinar quién controla el flujo del sistema.

Nivel	Sistema	Quién decide	Ejemplo concreto
0	Llamada única	El desarrollador	Responder una pregunta frecuente o extraer información puntual de un documento.
1	Cadena (*workflow*)	El desarrollador	Un pipeline secuencial fijo: extracción, validación y carga en ERP.

Nivel	Sistema	Quién decide	Ejemplo concreto
2	Enrutador	El modelo elige entre rutas fijas	Triaje automatizado: ticket severo, sprint inmediato; ticket bajo, backlog.
3	Agente con herramientas	El modelo (bajo presupuesto)	Depuración de un test: el agente decide qué archivos leer y cuándo detenerse.
4	Sistema multiagente	Orquestador + agentes especializados	Desarrollo y despliegue de una funcionalidad completa de punta a punta.

La tesis económica fundamental: El nivel de autonomía correcto es el mínimo que resuelve el problema de negocio, no el máximo que la tecnología permite. No implementes un Nivel 3 si un Nivel 1 soluciona el caso de uso. El diseño simple es más barato, predecible y fácil de auditar.

Mapeo de relación entre ambos marcos

Si se desea integrar la **Tabla de Arquitectura (0-4)** con el **Diagrama de Riesgo (N1-N5)**, la correspondencia de ejecución sería la siguiente:

1. **Agente/Sistema Entrada (Nivel 3/4 de la tabla):** El *Enjambre de Agentes* genera tareas complejas.
2. **Enrutamiento (Nivel 2 de la tabla):** El *Risk & Cost Router* actúa como un enrutador inteligente determinando la ruta fija (Carril Exprés, Asíncrono o Crítico).
3. **Ejecución y Control (N1 a N5 del diagrama):** Determina si el resultado requiere intervención directa del desarrollador/operador humano antes de su aplicación en producción.

Mapeo Integrado: Capa de Autonomía (0-4) vs. Capa de Gobernanza (N1-N5)

Ambos esquemas no se contradicen, sino que operan en **dos capas distintas del ciclo de vida de una tarea de IA**:

1. **Capa de Generación y Ejecución (Niveles 0 a 4):** Define *cómo piensa y actúa el sistema.* Un **Sistema Multiagente (Nivel 4)** o un **Agente con herramientas (Nivel 3)** genera las tareas o planes de acción.
2. **Capa de Control y Gobernanza (Niveles N1 a N5):** Define *cuánta libertad tiene el sistema para aplicar los cambios.* Un **Router de Riesgo (Nivel 2 de arquitectura)** intercepta cada tarea generada, mide su *Blast Radius* (Impacto financiero, tiempo y reversibilidad) y la clasifica en un carril de política de seguridad.

Flujo de Interacción

- **Generación:** Un **Enjambre de Agentes (Nivel 4 de la tabla)** coloca miles de tareas en la *Cola de Tareas.*
- **Evaluación:** El **Risk & Cost Router (Nivel 2 de la tabla)** actúa como un *Proxy/Middleware* de seguridad que decide la ruta evaluando el *Blast Radius.*
- **Despacho:**
  - Si la tarea del agente es cosmética (resumen, formato), baja a **N5/N4 (Carril Exprés)** y ejecuta como **Workflow Fijo (N1 de la tabla)** sin supervisión.
  - Si la tarea ejecuta acciones en API con estado pero reversibles, va a **N3 (Carril Asíncrono)** con política de *Rollback.*
  - Si la tarea implica escribir en Base de Datos de producción o transferencias monetarias (> $1K), el Router la frena en el **Muro de Validación (N2/N1)** hasta requerir aprobación humana (*Human-In-The-Loop*), independientemente de qué tan autónomo sea el agente que la creó.

Caso de estudio para analizar los niveles

Un equipo de ingeniería en Buenos Aires intentó automatizar por completo su soporte técnico mediante un enjambre de 7 agentes orientados a clasificar tickets, buscar documentación técnica, sugerir

parches de código y escalar incidentes críticos. Implementaron un Nivel 4 de autonomía sin haber dominado antes las bases del Nivel 1. Resultados tras tres meses en producción:

- **Costo de tokens:** $2.400 dólares al mes.
- **Tasa de acierto operativo:** 34 % (sin revisión humana).
- **Inversión en debugging:** 3 ingenieros asignados a tiempo completo para corregir fallos del sistema.
- **Satisfacción del cliente (CSAT):** Cayó un 12 %. **El error de diagnóstico:** Asumieron que añadir la complejidad de una arquitectura multiagente resolvería la complejidad intrínseca de su propio negocio. En producción ocurre lo contrario: la arquitectura multiagente añade fricción técnica, no la elimina. **La estrategia de corrección:**

1. **Paso 1 (Nivel 1):** Diseñaron un *workflow* determinista fijo para los 3 tipos de tickets más frecuentes.
2. **Paso 2 (Nivel 2):** Implementaron un enrutador basado en LLM para derivar solicitudes entre 5 categorías claras.
3. **Paso 3 (Nivel 3):** Agregaron un agente con acceso a herramientas de consulta para tickets de investigación profunda.
4. **Paso 4 (Nivel 4):** Reservaron el enjambre multiagente en producción exclusivamente para el 5 % de los tickets con mayor ambigüedad técnica.

Métricas tras seis meses de la corrección:

- **Costo de tokens:** Se redujo a $380 dólares al mes.
- **Tasa de acierto operativo:** Subió al 87 %.
- **Tiempo de ingeniería en debugging:** Reducido a 0.5 FTE.
- **Satisfacción del cliente (CSAT):** Incrementó un 8 %.

La decisión de nivel

Antes de delegar autonomía operativa en producción, tu comité técnico debe responder tres preguntas críticas:

1. **¿Sabemos con precisión cuánto gastamos al mes en tokens y qué servicios los consumen?**

Si la respuesta es negativa, tu arquitectura está en nivel 1. Detén el desarrollo agéntico y construye primero un *AI Gateway*. Un equipo de 4 a 6 ingenieros de plataforma dedicados a la observabilidad multiplica la eficiencia de 100 desarrolladores. No contar con visibilidad de costos multiplica el gasto operativo por un factor de 3 a 5.

2. **¿Qué porcentaje del código generado de forma autónoma llega a producción sin necesidad de retrabajo humano?**

Si la métrica es inferior al 40 %, estás inyectando deuda técnica acelerada en tus repositorios. *(Consúltese el Capítulo 11).*

3. **¿Tenemos un plan estratégico para transformar a nuestros ingenieros senior en orquestadores de sistemas?**

La transición no ocurre de manera orgánica. Requiere formación deliberada en diseño de sistemas distribuidos y control de flujo. *(Consúltese el Apéndice C: Roadmap de aprendizaje).*

Ejercicio para clasificar tus sistemas

Enunciado: Clasifica el nivel de autonomía (0 al 4) de los siguientes cuatro sistemas corporativos: - (a) Un servicio que recibe un ticket de soporte técnico y devuelve un resumen ejecutable en una sola línea. - (b) Un asistente que inspecciona el repositorio de código, reproduce un fallo de integración de forma local y propone un parche de corrección. - (c) Un flujo de agente automatizado que extrae datos estructurados de facturas en PDF, los valida contra reglas de negocio fijas y los carga en el ERP corporativo. - (d) Un sistema compuesto por un agente coordinador que descompone una migración de base de datos y reparte las tareas de refactorización entre agentes especialistas de código, bases de datos y testing.

Solución comentada:

- **(a) Nivel 0 (Llamada única):** El sistema realiza una única ejecución por ticket mediante API. No existen decisiones de flujo ni ramificaciones en tiempo de ejecución; es una clasificación simple.
- **(b) Nivel 3 (Agente con herramientas):** El modelo evalúa las herramientas disponibles en su entorno, decide autónomamente qué archivos examinar, cuándo ejecutar la suite de pruebas y en qué momento detener la iteración bajo un presupuesto de tokens controlado.
- **(c) Nivel 1 (Cadena / Workflow):** Las etapas del proceso son completamente secuenciales y deterministas. El uso de la palabra "agente" aquí es solo marketing; el flujo es un pipeline tradicional de software.
- **(d) Nivel 4 (Sistema multiagente):** Requiere descomposición dinámica de tareas en múltiples dominios técnicos en paralelo. Este nivel solo es viable si los agentes individuales de Nivel 3 operan de forma estable y aislada primero.

Lección de producción: El nivel de autonomía se selecciona basándose en la estructura técnica de la tarea, nunca por la ambición de los equipos de desarrollo.

Lo esencial del capítulo 1

La secuencia evolutiva 2017-2026: Pasamos de la arquitectura Transformer elemental al *few-shot learning*, la generación de código, la instrucción humana estructurada, el uso de herramientas de entorno y, finalmente, a los sistemas multiagente en producción.

- **Agente frente a chatbot:** Un agente cierra el ciclo operativo en producción porque percibe, razona y actúa sobre sistemas externos. Un chatbot tradicional solo ejecuta las dos primeras acciones.
- **El patrón ReAct equilibra el sistema:** Intercala pasos secuenciales de observación y razonamiento en cada ciclo para evitar que el agente ejecute planes desactualizados en entornos dinámicos.
- **El espectro consta de 5 niveles:** La regla de oro arquitectónica exige implementar siempre el nivel mínimo que resuelva el problema de negocio de forma predecible.

- **Saltar niveles destruye el ROI:** El caso de estudio de Buenos Aires demuestra cómo omitir la progresión de niveles genera sobrecostos en tokens y consume horas críticas de ingeniería en depuración.
- **El desplazamiento del costo operativo:** El cuello de botella de la ingeniería de software se trasladó de la fase de escritura de código a las fases de validación, gobernanza y auditoría automatizada.

El AI Engineer

Un rol emerge

En 2023, Shawn Wang ("swyx") publicó *The Rise of the AI Engineer*. No se trataba de una simple variación de títulos; emergía un profesional nuevo que no encajaba como ML Engineer (enfocado en entrenar modelos) ni como Data Scientist (orientado a analizar datos). Nacía el perfil encargado de construir aplicaciones reales sobre modelos ya entrenados. Entrenar un modelo base cuesta cientos de millones de dólares. Construir un producto encima cuesta una clave de API y la arquitectura correcta. Donde existe una asimetría económica y operativa de tal magnitud, emerge una nueva profesión de forma

Esta página está en la edición completa

Estás leyendo la muestra gratuita (página 34 de la obra).
Las 191 páginas incluyen los 21 capítulos, ejercicios resueltos y casos de estudio.

Ver dónde comprar el libro

Esta página está en la edición completa

Estás leyendo la muestra gratuita (página 35 de la obra). Las 191 páginas incluyen los 21 capítulos, ejercicios resueltos y casos de estudio.

Ver dónde comprar el libro

Esta página está en la edición completa

Estás leyendo la muestra gratuita (página 36 de la obra). Las 191 páginas incluyen los 21 capítulos, ejercicios resueltos y casos de estudio.

Ver dónde comprar el libro

Platform Engineering en la era agéntica

La plataforma como producto interno

Platform Engineering construye los caminos pavimentados por donde transitan los equipos. No es infraestructura: es la abstracción que convierte operaciones complejas en acciones de un clic. *Team Topologies* (Skelton y Pais 2019): la estructura debe minimizar la carga cognitiva. El equipo de plataforma absorbe esa carga. En la era agéntica, gana una responsabilidad adicional: exponer la IA como primitiva de primera clase. Si cada equipo negocia su propia clave, elige su modelo e

Esta página está en la edición completa

Estás leyendo la muestra gratuita (página 43 de la obra).
Las 191 páginas incluyen los 21 capítulos, ejercicios resueltos y casos de estudio.

Ver dónde comprar el libro

Esta página está en la edición completa

Estás leyendo la muestra gratuita (página 44 de la obra). Las 191 páginas incluyen los 21 capítulos, ejercicios resueltos y casos de estudio.

Ver dónde comprar el libro

Esta página está en la edición completa

Estás leyendo la muestra gratuita (página 45 de la obra). Las 191 páginas incluyen los 21 capítulos, ejercicios resueltos y casos de estudio.

Ver dónde comprar el libro

Invariantes de producción

Los 5 invariantes que todo sistema debe satisfacer

Un invariante va más allá de una "buena práctica": es una condición que siempre debe ser cierta, sin excepción. Si un invariante se viola, el sistema está en estado corrupto. En sistemas deterministas (código tradicional), los invariantes son fáciles: "el saldo de la cuenta no puede ser negativo", "el índice del *array* debe estar en rango". En sistemas agénticos (no deterministas, probabilísticos), los invariantes son más sutiles. Pero más críticos. Porque cuando fallan, el fallo es silencioso, gradual e invisible hasta que ya ha costado miles.

Esta página está en la edición completa

Estás leyendo la muestra gratuita (página 104 de la obra).
Las 191 páginas incluyen los 21 capítulos, ejercicios resueltos y casos de estudio.

Ver dónde comprar el libro

Esta página está en la edición completa

Estás leyendo la muestra gratuita (página 105 de la obra). Las 191 páginas incluyen los 21 capítulos, ejercicios resueltos y casos de estudio.

Ver dónde comprar el libro

Seguridad en sistemas multiagente

La superficie de ataque se multiplica

Cada agente es un nuevo vector. Cada herramienta es un nuevo permiso. Cada decisión autónoma es una nueva oportunidad de ser manipulado. En el software tradicional, la superficie de ataque es: *endpoints*, API, autenticación, sesiones. En sistemas multiagente, la superficie de ataque se multiplica:

1. Prompt injection: el usuario malicioso inyecta instrucciones a través de la entrada

Esta página está en la edición completa

Estás leyendo la muestra gratuita (página 142 de la obra). Las 191 páginas incluyen los 21 capítulos, ejercicios resueltos y casos de estudio.

Ver dónde comprar el libro

Esta página está en la edición completa

Estás leyendo la muestra gratuita (página 143 de la obra). Las 191 páginas incluyen los 21 capítulos, ejercicios resueltos y casos de estudio.

Ver dónde comprar el libro

Continúa con el libro completo

21 capítulos · ejercicios resueltos · casos de estudio
Arnés del agente, RACI en enjambres, invariantes de producción, deuda técnica agéntica, seguridad y costos.

Kindle · $9.99 USD

Todas las ediciones y tiendas

codigosintetico.specsolid.com